2026 第九届传智杯 · AI 创新应用挑战赛（个人赛）

低卡轻食点餐系统

基于意图驱动开发的全栈点餐 / 商家后台闭环
技术文档

作品名称	dika-light-meal-miniprogram
参赛形式	个人赛（1 人）
技术形态	微信原生小程序（TypeScript） + Native HTML/CSS/JS 展示端 + Node.js 原生 HTTP
代码仓库	https://github.com/as5827/dika-light-meal-miniprogram
文档版本	v1.1 · 2026 年 10 月

目录

一、选题与问题定义

1.1 场景与用户　1.2 三个具体痛点　1.3 目标与范围界定

二、产品方案与业务闭环

2.1 功能地图　2.2 订单闭环时序　2.3 关键界面

三、意图控制策略（AI 协作方法）

3.1 协作范式　3.2 上下文管理　3.3 Prompt 结构模板　3.4 三次迭代的 Prompt 链　3.5 纠偏与复核　3.6 约束清单

四、架构设计与工程实现

4.1 分层与职责　4.2 请求管线　4.3 数据一致性与原子写　4.4 订单状态机　4.5 对生成代码的约束

五、可复现性论证

5.1 提交策略　5.2 Prompt 归档　5.3 十分钟复现路径　5.4 验证证据　5.5 不可复现项声明

六、工程质量与安全

6.1 鉴权与会话　6.2 输入校验与金额可信　6.3 并发与文件一致性　6.4 风险清单

七、用户体验与产品精细度

八、局限与后续规划

附录

A 接口清单　B 数据模型　C Prompt 归档索引

阅读指引：本文档对应赛项评分维度组织——第三章对应「方法创新性与意图控制（20%）」，第四章对应「工程质量与架构设计（15%）」，第五章对应「过程可复现性与演进链路（25%）」，第二、七章对应「解题有效性与实用价值（25%）」「用户体验与产品精细度（15%）」。所有数据、命令与截图均来自本仓库真实运行结果。

一、选题与问题定义

1.1 场景与用户

轻食门店的典型经营形态是：菜品 SKU 不多（20–40 个）、但每天要改价格、估清（售罄）、换图；顾客以写字楼与校园周边为主，通过手机下单、到店自取或短途配送。这类门店通常没有预算去接一套 SaaS 点餐系统，一个「能改菜单 + 能收单 + 顾客能看到进度」的最小系统就足以覆盖 80% 的日常。

因此本作品服务两类角色：

角色	核心诉求	典型动作
顾客	快速看清今天有什么、热量多少、多少钱，下单后知道做到哪一步了	选规格 → 加购 → 填地址 → 下单 → 看进度
店主 / 店员	随时改价、估清、换图；新订单能听见看到，做完点一下	登录后台 → 改菜品 → 看订单 → 接单 → 完成

1.2 三个具体痛点

不是泛泛的「餐饮数字化」，而是三个可验证的具体问题：

1. **热量与价格信息不透明。**轻食客群对「多少卡」的敏感度高于普通快餐，但多数小店只在小程序图片里贴一张模糊的营养表。本作品把 `calorie` 作为菜品一级字段，点餐页与购物车实时累计总热量（`共 990 kcal`），并在后台可按菜品维护。
2. **估清靠嘴说。**「今天牛油果没了」通常靠店员口头告知，线上仍可下单。本作品把 `stock` 落到菜品上：库存 0 时点餐页直接置灰并显示「已估清」，接口层也会在下单时二次校验并返回 409。
3. **下单后进入信息黑箱。**顾客不知道商家看没看到单。本作品让订单状态从「待接单 → 制作中 → 已完成」单向推进，并把状态事件链同步到顾客端跟踪页（5 秒轮询），商家一点「接单」，顾客页面上的进度条就会走。

1.3 目标与范围界定

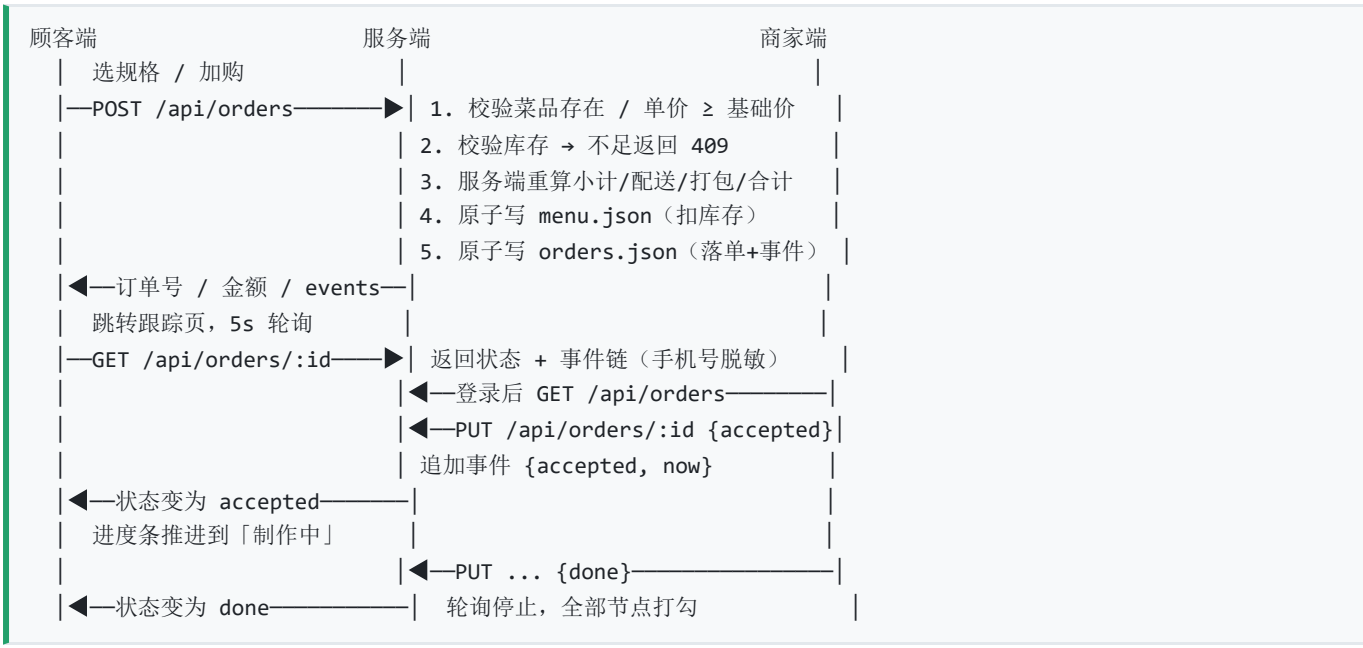
维度	本期做	本期不做（及原因）
交易	下单、金额重算、库存扣减、状态流转	支付、退款、发票——需要支付资质与回调联调，个人赛周期内无法真实验证
履约	商家接单 / 完成，顾客端可见	骑手调度、路径规划——本项目不涉及真实配送体系，故不虚构骑手信息
账号	商家单账号登录（Cookie 会话）	顾客注册体系——顾客以「订单号」作为凭证查询，降低使用门槛
部署	单机 Node 进程 + JSON 文件	容器化、多实例、CDN——与「零依赖」定位冲突，列入后续规划

二、产品方案与业务闭环

2.1 功能地图

端	模块	能力
顾客端 `/`	菜单浏览	8 个类目 / 28 道菜品；左侧导轨与右侧列表双向滚动联动；售罄置灰；骨架屏加载
	规格与购物车	多规格组（必选 / 可选）、加价项实时改单价、飞入动画、数量角标、热量合计
	地址与结算	地址簿增删改（localStorage）、配送费 ¥5 / 打包费 ¥2 / 满 ¥58 免配送、备注选填
	下单与跟踪	提交订单落库；跟踪页显示订单号、金额、进度条与状态时间轴，5 秒轮询自动刷新
商家端 `/admin`	登录鉴权	Cookie 会话（HttpOnly + SameSite）、连续 5 次失败锁定 60 秒、退出登录
	菜品管理	行内改名称 / 描述 / 价格 / 库存 / 热量；图片上传（dataURL → 落盘）；经营概览统计
	订单管理	最近订单列表（明细 / 地址 / 备注 / 金额）、接单、完成、取消

2.2 订单闭环时序



这条链路是本作品的「业务主干」：其余功能（改价、换图、估清）都是围绕它服务的输入。它同时满足三种视角——顾客看得见进度、店主点得动状态、评委点开链接能自己复现。

2.3 关键界面

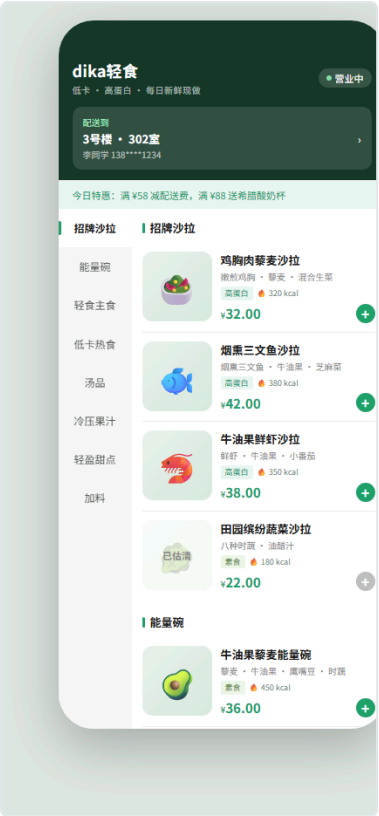


图 1　顾客端点餐页（移动端）



图 2　订单跟踪页（状态由服务端驱动）

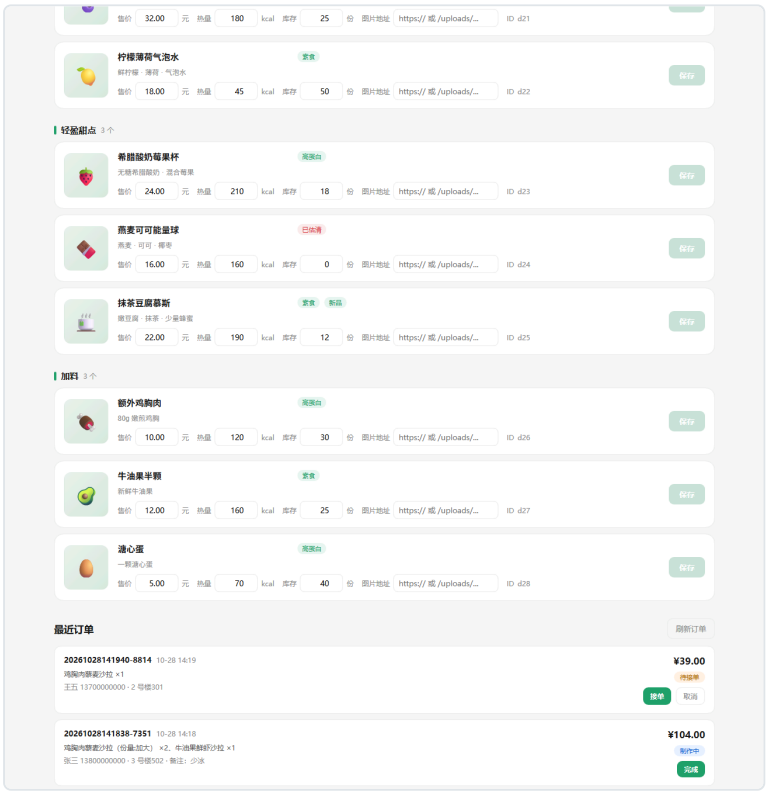


图 3　商家端后台：菜品管理与最近订单

三、意图控制策略（AI 协作方法）

本章回答一个问题：在「代码主要由 AI 产出」的前提下，人如何保证结果不走样、可维护、可复现。

3.1 协作范式：人定边界，AI 填实现

本项目采用「意图驱动 + 强约束」的协作方式，人的职责集中在四件事，AI 负责在这些边界内产出实现：

人的职责	具体做法	在仓库中的痕迹
定边界	先声明硬约束（零第三方依赖、Node ≥18、单文件前端、数据落 JSON），再让 AI 出方案	`prompts/01` 第 2 节、`README` 技术栈表
定验收	每个迭代先写「怎样算完成」，再写代码	`prompts/01` 的 A/B/C/D 交付物表
做取舍	对 AI 给出的备选方案逐条拍板，记录否决理由	`prompts/0x` 的「方案取舍」表
复核	不接受「看起来对」，必须给出状态码、日志或截图	`prompts/0x` 的「验证记录」表

3.2 上下文管理

AI 走样的主要原因不是能力，而是上下文给错。本项目在每次迭代开场都会显式投喂三类信息：

1. **硬约束**：如「服务端必须零第三方依赖，不能引入 npm 包」。这条约束直接否决了 SQLite、Express、JWT 库等一整套方案，避免了后期回退。
2. **现状事实**：如「`preview/index.html` 的提交按钮目前只做了前端校验并跳转，订单从未落库」。用代码事实替代模糊描述，AI 才不会重复实现已有能力。
3. **受影响文件**：只喂需要改的文件与相邻的接口定义，避免把整个仓库塞进上下文导致风格漂移。

> **反例（真实发生）：**在第一次建仓时，AI 用 PowerShell 把中文 JSON 作为命令行参数传给 `curl`，Windows PowerShell 5.1 按 GBK 编码传递参数，GitHub 收到非法 UTF-8 直接返回 400。定位后改为「UTF-8 文件 + `--data-binary @file`」一次通过。**这类问题的正确处理方式是把失败现象与原始响应体一起回灌给 AI，而不是只说「报错了」。**

3.3 Prompt 结构模板

凡是涉及多文件改动的任务，统一按下面的五段式组织，避免「帮我做个后台」这种开放式指令：

```
【角色】你是我这个零依赖 Node 项目的实现者，改动范围限定在 server/server.js 与 admin/index.html。
【约束】不引入任何第三方依赖；不改变现有接口的返回结构；错误统一 { code, message }。
【现状】后台目前没有任何鉴权，PUT /api/dishes/:id 可被匿名调用；前端靠隐藏按钮"假装"受限。
【目标】实现 Cookie 会话登录，受保护接口未登录返回 401，登录失败限速。
【验收】给出未登录 401、错误口令 401、正确登录 200、登出后 401 四组实测结果。
```

模板的价值在于把「验收标准」写进 Prompt 本身——AI 会主动围绕验收项组织实现，而不是堆功能。

3.4 三次迭代的 Prompt 链

仓库中 `prompts/` 目录按迭代归档，每条记录都包含需求原文、AI 拆解、方案取舍、接口契约与验证记录。三次迭代的核心决策如下：

迭代	关键决策	被否决的方案与理由
01 订单落库	JSON 单文件存储；服务端重算金额；下单扣库存	SQLite（`node:sqlite` 在 Node 18 不可用）；前端算价传后端（金额可篡改）；不扣库存（后台改库存成摆设）
02 登录鉴权	Cookie + 内存 Map 会话；口令 SHA-256 后定时安全比较；按 IP 限速	JWT（零依赖下要自己实现 HMAC 与过期校验，收益低）；Basic Auth（无法主动登出）；全局失败计数（一人锁死所有人）
03 状态驱动跟踪	落 events 事件链；5 秒轮询；终态停止	SSE / WebSocket（零依赖下握手与重连成本高，状态变化是低频事件）；用 `updatedAt` 推算时间轴（第二次改状态会覆盖第一次）

3.5 纠偏与复核

AI 的产出不直接进主干，必须过三道闸：

1. **语法闸**：`node --check server.js` 与内联脚本抽离后 `node --check`；
2. **行为闸**：用 `curl` 打真接口，核对状态码与响应体，覆盖正常路径 + 至少两条异常路径；
3. **视觉闸**：headless 浏览器截图，确认页面确实渲染成预期样子（防止「接口对但界面崩」）。

三道闸在 `prompts/` 的验证记录表里逐条留痕，这也是第五章「可复现性论证」的证据来源。

3.6 约束清单

约束	来源	对实现的直接影响
零第三方依赖	部署环境不可控，避免 `npm install` 失败	路由、静态服务、Cookie 解析、原子写全部手写；代价是功能密度受限
Node ≥ 18	评审可能用较老运行时	不使用 `node:sqlite` 等新 API
前端单文件	免构建、可直接双击查看/评审	无打包工具、无框架，样式与逻辑内联
金额以「分」为整数	浮点误差不可接受	全部金额字段整数存储，展示层转换

四、架构设计与工程实现

4.1 分层与职责

文件	行数	职责
`server/server.js`	518	HTTP 服务：路由分发、静态托管、会话鉴权、参数校验、原子读写、订单状态机
`preview/index.html`	1721	顾客端：菜单渲染、SKU、购物车、地址、结算、订单跟踪与轮询
`admin/index.html`	587	商家端：登录、菜品编辑、图片上传、订单列表与状态操作
`data/menu.json`	—	菜单 + 计费规则（唯一数据源，前后端共用）
`data/orders.json`	—	订单（运行时生成，已 gitignore，结构见 `orders.example.json`）
`miniprogram/`	25 个文件	生产端：微信原生小程序（TS strict），三页 + 服务层 + 购物车 store + 工程配置

分层原则：**服务端是唯一的业务真相。**前端只负责展示与交互，金额、库存、状态合法性全部由服务端裁决；前端计算仅用于即时反馈，不作为提交依据。

4.2 请求管线

```
request
  → 路由匹配（按 method + path 前缀，顺序敏感：精确匹配的 /api/orders 先于 /api/orders/:id）
  → 鉴权（仅受保护接口：PUT /api/dishes/*、POST /api/upload、GET /api/orders、PUT /api/orders/*）
  → 请求体读取（readBody 带体积上限：普通 64KB / 下单 256KB / 上传 8MB）
  → 参数校验（类型 + 范围 + 业务规则：菜品存在、单价 ≥ 基础价、库存充足）
  → 业务处理（重算金额 → 原子写菜单 → 原子写订单）
  → 统一响应（成功 { code:0, data }；失败 { code:, message }）
```

统一响应结构让前端可以用同一套 `if (!res.ok || json.code !== 0) throw new Error(json.message)` 处理所有接口，Toast 直接展示后端文案（如「「烟熏三文鱼沙拉」库存不足，仅剩 12 份」）。

4.3 数据一致性与原子写

JSON 文件最大的风险是「写一半崩溃 → 文件损坏 → 全部数据丢失」。本项目对所有持久化写入统一采用「临时文件 + rename」：

```
const writeOrders = (list) => {
  const tmp = ORDERS_FILE + '.tmp';
  fs.writeFileSync(tmp, JSON.stringify(list, null, 2), 'utf8');
  fs.renameSync(tmp, ORDERS_FILE);   // rename 在同一分区上是原子操作
};
```

下单时先写菜单（扣库存）再写订单，顺序不可颠倒：如果反过来，可能出现「订单已生成但库存没扣」的超卖；当前顺序最坏情况是「库存扣了但订单没落」，属于可接受的保守失败（少卖而非超卖），并已在限制说明中标注。

4.4 订单状态机

当前状态	允许流转到	触发方	副作用
`pending` 待接单	`accepted` / `canceled`	商家	追加事件；顾客端进度条推进
`accepted` 制作中	`done` / `canceled`	商家	追加事件；顾客端显示「制作中」
`done` 已完成	—（终态）	商家	追加事件；顾客端停止轮询
`canceled` 已取消	—（终态）	商家	追加事件；顾客端隐藏进度条

服务端显式拒绝把状态改回 `pending`，其余非法值一律 400。事件链 `events[]` 与状态同步追加，保证时间轴永远可还原，而不是靠单一 `updatedAt` 猜测。

4.5 对生成代码的约束

约束	落地方式
显式路由，不用中间件魔术	每个接口一个 `if (req.method === ... && url.startsWith(...))` 块，出错时能一眼定位
错误码约定	400 参数 / 401 未登录 / 404 不存在 / 409 冲突（库存）/ 413 过大 / 429 限速 / 500 服务端
输入一律当敌人	`asInt(v,min,max)` / `asStr(v,max)` 统一收口，禁止直接使用 `req.body` 字段
注释解释「为什么」	如 `// rename 在同一分区上是原子操作`、`// 首次运行时文件不存在`
前端不重复业务逻辑	前端算价仅用于展示；提交后以服务端返回的 total 为准

4.6 双端架构：小程序（生产）+ H5（展示）

同一个后端服务两个前端，是本项目对「可上架」与「可演示」这对矛盾的解法：

端	目录	职责与取舍
生产端	`miniprogram/`	微信原生小程序 + TypeScript strict。3 个页面（点餐 / 确认 / 跟踪），运行时依赖 0；上架后给真实顾客使用
展示端	`preview/`	H5 单文件，双击即开。牺牲微信登录与支付，换来「评委点开链接就能点单」，也是录屏演示的主界面
后台	`admin/`	改菜品、接单、完成；与两端共用同一套接口
后端	`server/`	10 个接口，两端共用；业务逻辑只写一遍，改一处两端同时生效

两端字段约定严格对齐：规格加价字段为 `deltaPrice`、计费规则来自 `menu.fees`、金额一律以「分」为整数传输——展示端验证通过的接口，小程序端无需改动即可复用。

上架的真实门槛不在代码：HTTPS 域名 + ICP 备案 + 餐饮类目资质（食品经营许可证）这三件套决定能否提交审核；微信登录与支付则是「要不要做真实交易」的产品决策。文档不回避这一点，见第八章。

五、可复现性论证

本章的目标是回答评委的四个追问：代码是不是一次性堆出来的？AI 协作过程是否真实？换台机器能不能跑起来？凭据在哪？

5.1 提交策略

仓库当前 **20 次提交**（含小程序端、Prompt 归档与本文档的提交），按「一个功能模块 / 一次 AI 对话 = 一次提交」推进，语义化前缀区分类型（`feat` / `docs` / `chore`），提交信息正文写明改动点与取舍：

```
85dc4f7 docs: 更新 README 与截图（后台鉴权、状态驱动跟踪页）
1c7b900 docs(prompts): 补充 02 鉴权与 03 状态驱动跟踪的迭代记录
0457862 feat(preview): 跟踪页轮询真实订单状态
32dc3c3 feat(admin): 登录遮罩与退出登录
135614d feat(server): 后台会话与接口鉴权 + 订单事件链 + 公开订单查询
c194a57 docs: 补充订单 API、数据模型、界面截图与限制说明
5491b6a feat(admin): 后台订单列表与状态流转
42f6c72 feat(preview): 结算页对接下单接口，跟踪页展示真实订单
405bd41 feat(server): 订单落库 API（创建/查询/状态流转）+ 下单扣库存
aaacec6 docs(prompts): 建立 AI 协作过程归档规范与首份记录
a83d090 chore: 统一换行符规范（.gitattributes）
...（更早 6 次为存量代码按模块导入）
```

> **诚实声明：**最早的 6 次提交是存量代码按模块导入，发生在 `prompts/` 机制建立之前，因此**没有对应的 Prompt 记录**。本文档与仓库均未对这些提交做追溯性补写——伪造的 Prompt 链在评审追问下必然崩塌，可信度应当押在真实可查的后续迭代上。

5.2 Prompt 归档

`prompts/` 目录的约束写在 `prompts/README.md` 中，单文件必须包含七段：原始需求、上下文、AI 拆解、方案取舍、关键 Prompt 与产出、验证记录、遗留问题；编号与提交顺序一致，并在提交信息中互相引用。

编号	迭代	归档内容要点
01	订单落库	存储/写入/金额/库存/文件入库 5 项决策与否决理由；接口契约；重启持久化验证
02	后台登录与鉴权	会话载体 3 选 1；Cookie 属性；定时安全比较；按 IP 限速；受保护接口清单
03	订单状态驱动跟踪	轮询 vs SSE/WebSocket；事件链建模；状态映射表；虚构内容清理

5.3 十分钟复现路径

```
# 1. 克隆（Node ≥ 18，无需 npm install）
git clone https://github.com/as5827/dika-light-meal-miniprogram.git
cd dika-light-meal-miniprogram

# 2. 启动
node server/server.js
#   dika轻食 server running at http://localhost:3000
#   点餐页  http://localhost:3000/preview/index.html
#   后台    http://localhost:3000/admin

# 3. 顾客端：选菜品 → 选规格 → 加购 → 结算 → 新增地址 → 提交订单
#    预期：跳转跟踪页，显示真实订单号与金额，并出现「订单已提交」事件

# 4. 商家端：打开 /admin → admin / dika2026 登录
#    预期：看到「最近订单」中刚才那一单，状态「待接单」

# 5. 点「接单」→ 切回顾客端跟踪页
#    预期：5 秒内进度条推进到「制作中」，时间轴新增一条「商家已接单，开始制作」

# 6.（可选）小程序端：微信开发者工具导入 miniprogram/，先 node server/server.js
#    详情 → 本地设置 → 勾选「不校验合法域名」，即可用同一份数据点单
```

5.4 验证证据

下列结果均来自文档写作当天的真实执行（服务端口 3992，curl + headless Edge）：

用例	预期	实测
未登录 `GET /api/orders`	401	401
未登录 `PUT /api/dishes/d1`	401	401
错误口令登录	401 + 文案	`{"code":401,"message":"账号或密码不正确"}`
正确口令登录	200 + Set-Cookie	`{"code":0,"data":{"user":"admin","expiresIn":28800000}}`
登出后 `GET /api/orders`	401	401
库存不足下单	409	`{"code":409,"message":"「烟熏三文鱼沙拉」库存不足，仅剩 12 份"}`
单价低于基础价	400	`{"code":400,"message":"「烟熏三文鱼沙拉」价格异常"}`
匿名查询订单	200 + 脱敏	`phone=136****5678`，events=1
接单后事件链	pending → accepted	events=2，顺序正确
重启后订单仍在	持久化	重启进程后 `GET /api/orders` 仍返回该订单

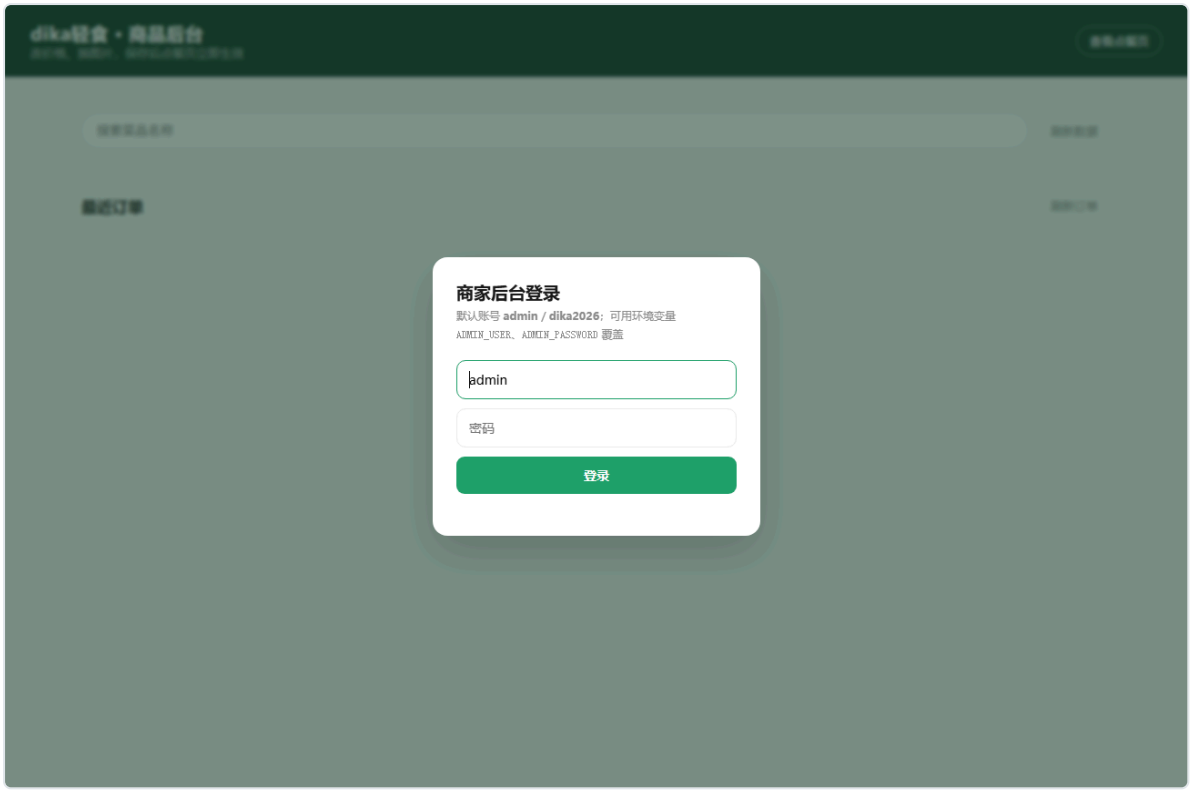


图 4　未登录访问后台时弹出登录遮罩（接口层同步返回 401）

5.5 不可复现项声明

- **AI 对话原文**：归档的是结构化摘要（需求原文 / 决策 / 验证），不是逐字对话记录；逐字记录体量过大且包含环境噪声。
- **初始 6 次提交**：无 Prompt 记录，已在 5.1 声明。
- **支付与配送**：未接入，任何「已支付」「骑手已接单」类信息都不应出现在演示中——本项目已主动删除原型的虚构骑手卡片。

六、工程质量与安全

6.1 鉴权与会话

措施	实现
会话载体	服务端内存 `Map<token, {user, expiresAt}>`，令牌 `crypto.randomBytes(24).toString('hex')`
Cookie 属性	`HttpOnly`（挡 XSS 读取）+ `SameSite=Lax`（挡跨站提交）+ `Path=/` + `Max-Age=28800`
口令比对	双方先 SHA-256 再 `crypto.timingSafeEqual`，避免长度与前缀的时序泄漏
暴力破解	按 IP 计数，连续 5 次失败锁定 60 秒，返回 429 并给出剩余秒数
会话续期	滑动续期：每次命中把过期时间推到 8 小时后

6.2 输入校验与金额可信

所有写接口遵循「先收口、再校验、后落库」：

```
const asInt = (v, min, max) =>
  typeof v === 'number' && Number.isInteger(v) && v >= min && v <= max ? v : null;
const asStr = (v, max) => (typeof v === 'string' ? v.trim().slice(0, max) : '');
```

支付相关字段（`unitPrice`、`qty`）只被当作「输入」而非「结果」：小计、配送费、打包费、合计、总热量全部由服务端基于菜单与计费规则重算；`unitPrice` 仅允许 ≥ 菜品基础价（规格只能加价）。

6.3 并发与文件一致性

当前写入路径是 `read → 改 → write`，多请求并发下单存在竞态窗口（两个请求可能读到同一份库存）。单机演示场景可接受，已在限制中标注；后续方案是文件锁（`fs.open` + 独占标志）或迁移 SQLite 事务。

6.4 风险清单

风险	等级	当前处置
默认口令 `admin/dika2026`	中	可通过环境变量覆盖；启动日志与登录框均提示
会话存内存，重启全失效	低	演示可接受；已列入规划
订单号可枚举（时间戳 + 4 位随机）	中	手机号已脱敏；后续提高随机熵或加签名
并发扣库存竞态	中	单机演示可接受；规划加锁
Cookie 未加 `Secure`	低	本地 http 调试无法设置；部署 HTTPS 时补

七、用户体验与产品精细度

场景	设计
浏览菜单	左侧导轨与右侧列表双向联动（滚动联动 + 点击平滑定位），解决长列表迷路；骨架屏避免白屏闪烁
选规格	必选组未选满时「加入购物车」置灰；加价项实时改单价，避免结算时价格跳变
加购反馈	飞入购物车动画 + 数量角标 + 底部结算栏，三重视觉确认
提交订单	按钮进入「提交中…」并禁用，防重复提交；失败 Toast 展示后端原文，库存类错误自动刷新菜单
跟踪页	进度条节点带呼吸动画指示当前状态；时间轴倒序、最新一条加粗；终态自动停止轮询
后台	行内编辑，改动即高亮 dirty，保存后按钮短暂显示「已保存」；401 统一回到登录态而不是静默失败
空态 / 错误态	菜单加载失败提供「重新加载」；订单为空提示「还没有订单，去点餐页下一单试试」

八、局限与后续规划

局限	规划
小程序端已实现三页闭环，但未接微信登录与支付，顾客以「订单号」作为凭证	接 `wx.login` + code2session 绑定 openid；接微信支付商户号与回调
无支付、无退款、无配送回调	接入支付沙箱与配送状态回调，让状态由外部事件驱动而非人工点击
JSON 文件存储，历史订单无分页	迁移 SQLite（Node ≥ 22 的 `node:sqlite`）并保留当前接口结构
会话与限速均为进程内存	持久化会话表 + 基于 Redis 的限速（或退化为文件计数）
无自动化测试	把 `prompts/` 里的 curl 用例固化为 `test/api.test.js`，用 `node --test` 运行

附录

A 接口清单

方法	路径	权限	说明
POST	`/api/login`	公开	登录，写 HttpOnly Cookie；失败 401，限速 429
POST	`/api/logout`	公开	销毁会话并清 Cookie
GET	`/api/session`	公开	返回当前登录用户，未登录 401
GET	`/api/menu`	公开	菜单 + 计费规则
POST	`/api/orders`	公开	下单：校验 → 重算金额 → 扣库存 → 落库
GET	`/api/orders/:id`	公开	订单跟踪，手机号脱敏，含事件链
PUT	`/api/dishes/:id`	登录	更新菜品价格/库存/热量/图片/名称/描述
POST	`/api/upload`	登录	图片上传（dataURL，≤2MB）
GET	`/api/orders`	登录	订单列表（倒序，limit ≤ 200）
PUT	`/api/orders/:id`	登录	状态流转，追加事件

B 数据模型

```
// data/menu.json
{
  "categories": [{ "id": "c1", "name": "招牌沙拉" }],
  "dishes": [{
    "id": "d1", "cat": "c1", "name": "鸡胸肉藜麦沙拉",
    "desc": "嫩煎鸡胸 · 藜麦 · 混合生菜",
    "price": 3200,          // 单位：分
    "calorie": 320,         // 单位：kcal
    "image": "", "emoji": "🥗", "stock": 20, "tags": ["高蛋白"],
    "specs": [{ "id": "g1", "name": "份量", "required": true,
                "options": [{ "id": "o1", "name": "标准", "delta": 0 }] }]
  }],
  "fees": { "deliveryFee": 500, "packFee": 200, "freeDeliveryThreshold": 5800 }
}

// data/orders.json（运行时生成，结构见 data/orders.example.json）
[{
  "id": "20261028120304-4821",
  "status": "pending",                        // pending | accepted | done | canceled
  "items": [{ "dishId": "d1", "name": "鸡胸肉藜麦沙拉", "specDesc": "份量:加大",
              "unitPrice": 3200, "qty": 2, "calorie": 320, "amount": 6400 }],
  "address": { "name": "张三", "phone": "13800000000", "building": "3 号楼", "room": "502" },
  "remark": "少冰",
  "subtotal": 6400, "deliveryFee": 0, "packFee": 200, "total": 6600,
  "totalCalorie": 640, "itemCount": 2,
  "events": [{ "status": "pending", "at": "2026-10-28T04:03:04.000Z" }],
  "createdAt": "2026-10-28T04:03:04.000Z", "updatedAt": "2026-10-28T04:03:04.000Z"
}]
```

C Prompt 归档索引

编号	文件	主要决策
01	`prompts/01-订单落库.md`	JSON 单文件存储、服务端重算金额、下单扣库存、订单文件不入库
02	`prompts/02-后台登录与鉴权.md`	Cookie + 内存会话、定时安全比较、按 IP 限速、公开订单查询与脱敏
03	`prompts/03-订单状态驱动跟踪页.md`	事件链替代 updatedAt、5 秒轮询、终态停止、清理虚构内容

文档所述功能与数据均可在仓库中复核：`https://github.com/as5827/dika-light-meal-miniprogram`